

BA820 – Project M5

- **Project Title:** Code Trends, Quantified: Mapping the Programming Language Ecosystem
- **Section and Team Number:** B1 Team 14
- **Student Name:** Drishti Chulani, Arshdeep Singh Oberoi, Ahrar Karim, Vishesh Goyal

1. M5 Contribution Table

Team Member	Concrete Technical Tasks (Plain English)	Concrete Method Usage	Evidence (GitHub Links)
Vishesh Goyal	Developed a technical "fingerprinting" system to group languages by their technical ecosystem. Architecture-Market Alignment: Measured whether a language's technical design actually predict	Applied TF-IDF Vectorization on file extensions; performed Hierarchical Clustering with Cosine Similarity; validated using Adjusted Rand Index (ARI) .	Vishesh Goyal's Individual M2 and M4 notebooks
Drishti Chulani	Deep Feature Extraction: Engineered a multi-layer neural network to strip away "Volume Bias" (legacy giants like Java) and isolate the underlying structural DNA of emerging technologies.	Deep Representation Learning: Symmetric Autoencoder (4-16-8-2-8-16-4) using Keras. Non-Linear Manifold Learning: UMAP projection and K-Means clustering on the 2D Latent Space to identify "Hidden Titans."	Drishti Chulani's Individual M2 and M4 notebooks
Arshdeep Singh Oberoi	Quantified the "Efficiency Gap" to distinguish professional enterprise utility from social media hype and identify "Ghost" languages.	Engineered Visibility-to-Employment Ratios ; performed DBSCAN robustness checks and Silhouette k-selection on log-transformed data.	Arshdeep Singh Oberoi's Individual M2 and M4 notebooks
Ahrar Karim	Built a predictive early-warning classification engine to calculate the statistical probability of a language becoming obsolete.	Random Forest Classification; Hyperparameter tuning (18-config grid); Apriori Association Rules; Threshold Sensitivity Analysis.	Ahrar Karim's Individual M2 and M4 notebooks

**Note: Full Project Contribution Table in the Appendix*

2. Problem Statement & Focus

The modern programming language ecosystem is highly saturated, with over 4,000 active and historical technologies competing for developer adoption. For technical leadership, choosing a foundational tech stack is a high-stakes gamble: selecting a language that ultimately fails results in millions of dollars in technical debt and forced infrastructure rewrites. With our historical data revealing that roughly 97% of recorded languages eventually become inactive "Ghosts," the business problem is clear: How can organizations safely identify resilient, long-term technology investments in an ecosystem dominated by noise?

The Analytical Problem: The "Hype Trap" and Volume Bias In earlier phases of this project, we attempted to map the market using traditional distance-based clustering (K-Means) on surface-level metrics like GitHub stars, Wikipedia views, and job postings. However, we hit an analytical wall. We discovered that raw volume is a lagging indicator of success. Massive, 30-year-old languages blinded our models, while social media buzz created a "False Hype" echo chamber that rarely converted into actual enterprise jobs. We realized that traditional descriptive analytics cannot answer our core business problem because they only measure past popularity, not future viability.

Our M5 Focus: From Descriptive Volume to Predictive Resilience To solve this, our final integrated pipeline completely shifts the analytical focus. We no longer ask, "How big is this language today?" Instead, we ask, "Does this language have the structural DNA to survive tomorrow?" Our focus is entirely dedicated to filtering out market noise to predict long-term industrial resilience. To achieve this, we narrowed our scope to four highly specialized objectives:

1. **Decoupling Architecture from Hype:** Proving mathematically whether a language's technical design (its file extensions and ecosystem) actually drives its market adoption, or if the two are entirely disconnected.
2. **Extracting Latent Market DNA:** Using deep dimensionality reduction to strip away "volume bias" and isolate the underlying structural blueprints of emerging technologies, allowing us to find "Hidden Titans" before they peak.
3. **Quantifying the Efficiency Gap:** Engineering strict ratios to mathematically separate internet trends from genuine enterprise utility.
4. **Predicting the "Ghost" Horizon:** Transitioning from historical look-backs to a predictive classification engine capable of calculating the exact statistical probability of a language's survival based on its current administrative vital signs.

By focusing on these four pillars, this report provides a data-driven blueprint for navigating the "Adoption Chasm" and making secure technology investments for 2026 and beyond.

3. EDA & Preprocessing: Updates

No new EDA or preprocessing steps were required, just combined all of the M4 analyses

4. Analysis & Experiments

To transition from the descriptive baselines of Milestone 3 to a predictive, structural understanding of the programming language market, our team engineered a unified pipeline. In Milestone 4, we realized that raw volume metrics were masking the true viability of technologies. To solve this, each team member developed a specialized analytical "lens." Together, these four experiments form our M5 Master Pipeline, sequentially filtering market noise to identify resilient technology investments.

Below is the detailed breakdown of each experiment and the specific technical contributions of the team members.

Experiment 1: Technical Fingerprinting & The Market Mismatch

Lead Contributor: Vishesh Goyal

Our first hypothesis was that a language's underlying technical architecture heavily dictates its market adoption. To test this, Vishesh moved away from volume-based metrics and focused strictly on the "tools" a language uses: its file extensions.

- **Methodological Execution:** Because file extension data is highly sparse and binary, traditional Euclidean distance metrics fail. We engineered a technical fingerprinting system by applying **TF-IDF (Term Frequency-Inverse Document Frequency) Vectorization** to the extensions, treating them as technical tokens. **Hierarchical Clustering using Cosine Similarity**, which groups languages based on the orientation of their technical vectors rather than their magnitude.
- **Key Findings & Validation:** To see if technical design correlated with popularity, these new technical clusters were compared against the M3 community demand clusters. The **Adjusted Rand Index (ARI)** was utilized to measure alignment. The result was an ARI near 0.0, empirically proving a massive "Technical-Market Mismatch." This crucial finding demonstrated that a language's design alone cannot predict its market success, necessitating the next phase of the pipeline.

Experiment 2: Extracting Latent Market DNA

Lead Contributor: Drishti Chulani

Having ruled out technical architecture as a reliable proxy for success, we needed to fix the "volume bias" inherent in our earlier models. Massive industry standards like Python and Java were skewing the feature space, masking the emergence of high-potential new languages.

- **Methodological Execution:** The linear K-Means clustering was replaced with a **Deep Symmetric Autoencoder (4-16-8-2-8-16-4 architecture)** built in Keras to compress 4 correlated metrics into a 2D latent bottleneck. To ensure the model learned meaningful

structures, an "Active Market Filter" was applied first, removing thousands of dormant ghost languages that were acting as pure mathematical noise. By forcing the remaining 953 active languages through a two-node bottleneck layer, the neural network successfully discarded raw volume and learned the non-linear relational structures—the "Latent DNA.."

- **Key Findings & Validation:** The Autoencoder successfully converged, optimizing the Mean Squared Error (MSE) loss down to 0.08. When this latent space was projected into 2D using **UMAP (Uniform Manifold Approximation and Projection)**, it revealed a **62% archetype shift** compared to the M3 baseline. This experiment successfully isolated the "Hidden Titans"—languages that currently lack the historical job volume of Java but possess an identical structural blueprint.

Experiment 3: Quantifying the Efficiency Gap

Lead Contributor: Arshdeep Singh Oberoi

To validate whether the "Hidden Titans" identified by the Autoencoder were actually being adopted by the industry or were just internet trends, an experiment was designed to measure the conversion rate between cultural buzz and enterprise adoption.

- **Methodological Execution:** Composite **Visibility-to-Employment Ratios** were engineered on log-transformed data. To ensure these groupings were not just artifacts of the standard K-Means algorithm, rigorous stability and robustness checks were conducted using **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) and **Silhouette k-selection**.
- **Key Findings & Validation:** The "Adoption Chasm" was successfully mapped. This analysis mathematically isolated "Ghost Languages" (technologies with high GitHub visibility but near-zero job conversion) from "Silent Workhorses" (technologies with low stars but high enterprise demand). Validation tests confirmed that the clusters were structurally sound regardless of the algorithm used, providing a concrete economic reality check to apply to the latent archetypes.

Experiment 4: Predictive Survival Modeling

Lead Contributor: Ahrar Karim

The final experiment shifted the pipeline from diagnosing the present to forecasting the future. In M3, it was descriptively identified that roughly 97% of recorded programming languages eventually become inactive. For M5, an early-warning engine was built to predict *when* a language would cross that threshold.

- **Methodological Execution:** This was framed as a binary classification problem (Active vs. Inactive). To handle the severe class imbalance, a comprehensive 4-model comparison (**Random Forest, Logistic Regression, KNN, and Decision Trees**) was conducted using an 18-config hyperparameter tuning process. To ensure the definition of "Active" was not arbitrary, a **Threshold Sensitivity Analysis** was performed, testing variants at ≤ 3 , ≤ 5 , and ≤ 8 years of inactivity, proving the results were mathematically robust across all definitions.
- **Key Findings & Validation:** Random Forest emerged as the superior model. Furthermore, **Apriori association rules** were utilized to extract actionable business logic. The strongest finding was that a language possessing both a registered domain and an active GitHub

presence yields a **Lift of 10.67x**. This specific insight provides stakeholders with the exact "Vital Signs" needed to predict if an emerging technology will survive long enough to yield a return on investment.

5. Findings & Interpretations

The integration of our four analytical lenses yielded a singular, cohesive conclusion: **Surface-level popularity is a lagging indicator of a programming language's success, and relying on it leads to high-risk technology investments.** By filtering out market noise and looking at the structural DNA of the ecosystem, we established three primary findings that directly answer the CTO's Dilemma of how to choose resilient tech stacks for 2026.

Finding 1: The "Technical-Market Mismatch" and the Hype Trap

Our initial assumption was that a language's technical design (what it is built to do) would naturally drive its market adoption. The technical fingerprinting analysis completely disproved this. By achieving an Adjusted Rand Index (ARI) of 0.0 between technical clusters and community clusters, we found that **technical architecture is entirely decoupled from social popularity**. This finding was contextualized by the Visibility-to-Employment ratios, which mapped the "Adoption Chasm." We discovered that social signals (like GitHub stars and Wikipedia views) frequently create a "False Hype" echo chamber. Many languages achieve massive cultural visibility but fail to cross the chasm into enterprise utility (actual job postings).

- **Interpretation:** Stakeholders must stop treating GitHub stars as a proxy for industrial readiness. A language can be technically sound and highly visible on social media, yet still completely lack the enterprise backing required to make it a safe infrastructure investment.

Finding 2: The "Hidden Titans" Lie Beneath the Volume Bias

Because traditional K-Means clustering relies on linear distance, our earlier models were blinded by the sheer volume of "Titan" languages like Java, Python, and SQL. Emerging technologies were incorrectly lumped into "Speculative" or "Niche" categories simply because they had not been in the market long enough to accumulate millions of job postings.

The Symmetric Autoencoder solved this by compressing the metrics into a latent blueprint. The resulting 62% archetype shift proved that the majority of the market is misclassified by traditional volume counting. The Autoencoder successfully identified **"Hidden Titans"**—languages like Kotlin, TypeScript, and Solidity.

- **Interpretation:** These languages may not yet have the raw historical job counts of a 30-year-old language like C++, but they possess the exact same structural "DNA" and balance of community investment to enterprise utility. For a CTO, these Hidden Titans represent the most lucrative, forward-looking investments: they are mature enough to be safe, but early enough to provide a competitive edge.

Finding 3: The "Vital Signs" of Obsolescence

Identifying a good language is only half the battle; avoiding a dying one is just as critical. With 97% of recorded languages eventually becoming inactive "Ghosts," the predictive classification engine provided the statistical guardrails for our investments.

The Random Forest model and Apriori association rules extracted the exact "Vital Signs" of survival. We found that complex technical features matter far less for survival than basic infrastructure maintenance. Specifically, a language that maintains both a registered web domain and an active GitHub repository yields a 10.67x Lift in its probability of remaining active.

- **Interpretation:** Obsolescence is rarely a sudden technical failure; it is a gradual community abandonment. If an enterprise is evaluating an emerging, niche technology, the first due-diligence check should not be its code execution speed, but rather the administrative pulse of its maintainers. If the domain registration lapses or repository commits stall, the language is entering the "Ghost Zone," regardless of its previous hype.

The Final Takeaway

The programming language ecosystem is not a meritocracy of technical design; it is an economy of structural resilience. To build a tech stack that survives the next decade, decision-makers must ignore the "False Hype" of internet popularity, utilize latent modeling to find the "Hidden Titans" before they peak, and constantly monitor the administrative "Vital Signs" to avoid the 97% of technologies destined to become Ghosts.

6. Appendix

6.1. Shared GitHub Repository (Required)

- 6.1.1. GitHub Repository Link: [GitHub Link](#)
- 6.1.2. GitHub Link to M5 Notebook: [M5 Notebook](#)
- 6.1.3. Vishesh's GitHub Folder: [VG Link](#)
- 6.1.4. Drishti's GitHub Folder: [Drishti Link](#)
- 6.1.5. Arshdeep's GitHub Folder: [Arshdeep Link](#)
- 6.1.6. Ahrar's GitHub Folder: [Ahrar Link](#)
- 6.1.7. Final Presentation Slide Deck Link: [Slides Link](#)

6.2. Contribution Table

Team Member	Technical Contributions	Methods & Tools Used	Presentation Slides
Vishesh Goyal	• Community clustering (silhouette-based k selection) • Technical baseline modeling (Top-N one-hot encoding + KMeans) • Weighted Bag-of-Words clustering (TF-IDF + cosine similarity + Hierarchical clustering) • ARI alignment analysis across clustering strategies • Association rule mining on extensions	KMeans; AgglomerativeClustering; TF-IDF; PCA; UMAP; ARI	1, 2, 3, 4, 5
Drishti Chulani	• Active Market data filtering • Autoencoder architecture design and training • Deep latent-space clustering • Archetype shift identification and interpretation	Keras Autoencoder; KMeans; UMAP; StandardScaler	6, 7
Arshdeep Singh Oberoi	• Visibility and employment score engineering • Log transformations for skew correction • Elbow and Silhouette-based k selection • Ghost Language identification • DBSCAN robustness validation • Cluster stability analysis	KMeans; DBSCAN; PCA; StandardScaler	8, 9, 12
Ahrar Karim	• Unified preprocessing pipeline development • Baseline PCA + KMeans recap • Random Forest modeling with 18-configuration hyperparameter tuning • Four-model comparative evaluation (LR, KNN, DT, RF) • Sensitivity analysis (three threshold variants) • Apriori association rule mining	RandomForest; LogisticRegression; KNN; DecisionTree; Apriori; PCA	10, 11

6.3. Generative AI Disclosure

I used Generative AI tools to support the writing and presentation of this Milestone 5 report, while ensuring that the analysis, code execution, figures, and numerical results came directly from my own notebook workflow. Specifically, I used ChatGPT (OpenAI) and Gemini (Google) to help organize the report in the required template format, improve clarity of explanations, and make technical interpretations easier to read.

Generative AI was used primarily for structuring sections, refining wording, improving transitions between ideas, and polishing figure captions so that the narrative remained consistent and easy to follow. It was also used to brainstorm alternative ways to communicate the same insight and to sanity-check whether my conclusions were written clearly for a non-technical audience. Any suggested text was carefully reviewed and edited by me to ensure it accurately reflected the notebook outputs and the methods used.

I did not use Generative AI to generate or fabricate data, results, metrics, tables, or figures, nor to alter or “improve” model performance. I also did not use it to replace the analytical process or to produce final outputs without verification. All computations (e.g., clustering results, ARI comparisons, autoencoder outputs, ghost-language analysis, survival prediction, and association rules) were produced from the notebook, and I verified that every claim included in the report matched the results printed or visualized in the notebook.

Link to ChatGPT: [*ChatGPT Link*](#)

Link to Claude: [*Claude Link*](#)

Link to Gemini: [*Gemini Link*](#)

6.4. Special Notes

We used ChatGPT to paraphrase and create the report as this was not a mandatory part of the submission, but we made the contribution tables ourselves as it was stated in the submission rubrics